DAKAR INSTITUTE OF TECHNOLOGY

Licence 3 — Data Engineering

Année académique 2025 – 2026

RAPPORT DE PROJET

Projet ETL & Data Warehouse

MACROBUS

Pilotage des Ventes et de la Performance

Membres du Groupe

Minko Moukila Cagil Chris Armand • Ossiala-Ngoli Syl Erwann Cohenn • Mohamadou Yaya Balarabé
Bazoungoula Lasie-Havila • Rita Marie Chambaz • Mouhamadou Falilou DIAGNE • Yves Chadrac Kate

Module : Data Engineering — Mars 2026

Sommaire

I.	Introduction & Contexte du Projet	3
II.	Compréhension des Outils	4
III.	Architecture du Système	6
IV.	Infrastructure Docker	7
V.	Base de Données Source (OLTP)	8
VI.	Entrepôt de Données — Schéma en Étoile	10
VII.	Flux ETL avec SSIS	12
VIII.	Génération des Données de Test	16
IX.	Validation Fonctionnelle de l'Entrepôt	18
X.	Tableau de Bord Power BI	20
XI.	Conclusion	22

I. Introduction & Contexte du Projet

1.1 Présentation du Projet

Ce rapport présente le travail réalisé dans le cadre du module Data Engineering en Licence 3 au Dakar Institute of Technology. Notre groupe a conçu et développé un système complet de Business Intelligence pour l'entreprise fictive MacroBus, spécialisée dans la commercialisation de bus et autocars à l'échelle internationale.

1.2 Présentation de MacroBus

MacroBus est une entreprise multinationale présente sur trois grandes zones géographiques. Elle dispose de plusieurs filiales réparties à travers le monde et commercialise trois catégories de produits phares :

- Voyage : autocars grand tourisme et interurbains longue distance
- Urbain : bus pour le transport en commun urbain (citybuses)
- Transport Scolaire : véhicules adaptés aux trajets scolaires

Les filiales opérationnelles sont : MacroBus Sénégal, MacroBus France, et MacroBus Canada, couvrant les territoires Afrique, Europe et Amérique.

1.3 Problématique

> Comment centraliser et analyser efficacement des données de ventes dispersées dans un système transactionnel, afin de produire des indicateurs de performance fiables et actionnables pour la direction de MacroBus ?

1.4 Objectifs

- Concevoir une base de données source transactionnelle (OLTP) représentative de l'activité MacroBus
- Modéliser un entrepôt de données (Data Warehouse) en schéma en étoile
- Développer un flux ETL complet avec SQL Server Integration Services (SSIS)
- Alimenter l'entrepôt avec 10 000 commandes réalistes couvrant 2003–2005
- Créer un tableau de bord Power BI interactif pour le pilotage des ventes

1.5 Technologies Utilisées

Technologie	Outil	Rôle dans le Projet
SQL Server 2022	Moteur de BDD Microsoft	Héberge la source et le DW
Docker Desktop	Conteneurisation	Infrastructure portable

SSMS	SQL Server Management Studio	Exécution des scripts SQL
SSIS / SSDT	Visual Studio 2022	Développement du flux ETL
Power BI Desktop	Outil BI Microsoft	Visualisation & Dashboard
T-SQL	Scripts SQL	Création & alimentation des BDD

II. Compréhension des Outils Microsoft BI

Avant de se lancer dans la réalisation technique, notre équipe a pris le temps de bien maîtriser chaque outil. La distinction entre les différents composants de l'écosystème Microsoft est fondamentale pour éviter toute confusion.

2.1 SQL Server — Le Moteur de Données

SQL Server est le système de gestion de base de données (SGBD) de Microsoft. C'est ici que vivent physiquement toutes les données du projet. On peut faire une analogie directe avec PostgreSQL : les deux sont des SGBD relationnels, mais SQL Server utilise le dialecte T-SQL (Transact-SQL) au lieu du PL/pgSQL.

SQL Server n'est pas qu'un simple entrepôt de tables. Il vient avec une suite d'outils intégrés indispensables pour le projet :

- SSIS (Integration Services) : l'outil ETL — extrait, transforme et charge les données
- SSRS (Reporting Services) : génération de rapports (remplacé ici par Power BI)
- SSAS (Analysis Services) : création de cubes OLAP pour analyses avancées

Écosystème PostgreSQL	Écosystème SQL Server
PostgreSQL Server	SQL Server (moteur)
pgAdmin / DBeaver	SSMS (interface GUI)
Talend / Pentaho	SSIS via SSDT (ETL)
PL/pgSQL	T-SQL (dialecte)

2.2 SSMS — L'Interface de Contrôle

SQL Server Management Studio (SSMS) est à SQL Server ce que pgAdmin est à PostgreSQL : l'interface graphique de contrôle. C'est ici qu'on exécute les scripts SQL pour créer les bases, gérer les tables, et vérifier que les données sont correctement chargées.

Dans notre projet, SSMS a servi à deux phases distinctes :

- Création (le contenant) : exécution de MacroBus_Source.sql et MacroBus_DW.sql pour créer la structure vide
- Vérification (le contenu) : requêtes SELECT pour valider le chargement ETL

> ⚠ Ne pas confondre SSMS (interface d'exécution SQL) et SSDT/SSIS (outil de développement ETL). Ce sont deux logiciels distincts avec des rôles complémentaires.

2.3 SSIS / SSDT — L'Usine ETL

SQL Server Data Tools (SSDT) est une extension de Visual Studio. C'est l'environnement de développement du pipeline ETL. Plutôt que d'écrire du code SQL pour déplacer les données, on dessine des flux visuels reliant une source à une destination, avec des transformations intermédiaires.

Trois concepts fondamentaux structurent un package SSIS :

- Control Flow (Flux de contrôle) : définit l'ordre d'exécution des tâches — les dimensions sont chargées avant la table de fait
- Data Flow (Flux de données) : le tuyau de transformation — Source OLE DB → Colonne dérivée → Recherche → Destination OLE DB
- Connection Managers (Gestionnaires de connexion) : les 'prises' vers MacroBus_Source et MacroBus_DW

III. Architecture du Système

3.1 Vue d'Ensemble

Le projet s'articule autour d'une architecture classique de Business Intelligence en trois couches, garantissant séparation des responsabilités, performances analytiques optimisées et intégrité des données.

> OLTP (MacroBus_Source) → ETL / SSIS (Visual Studio) → DW (MacroBus_DW) → Power BI Dashboard

Couche	Technologie	Rôle
Source (OLTP)	SQL Server + Docker	Données transactionnelles brutes
ETL	SSIS / SSDT	Extraction, Transformation, Chargement
Analytique (DW)	SQL Server DW	Données structurées en étoile
Visualisation	Power BI Desktop	Dashboard & KPIs décisionnels

3.2 Arborescence du Projet

Le projet est organisé de manière à être facilement reproductible par tous les membres de l'équipe. La structure est la suivante :

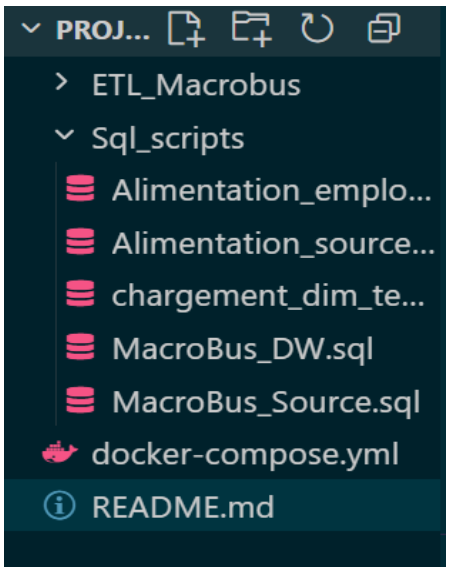

Figure 1 — Arborescence du projet dans VS Code

Les dossiers principaux sont : ETL_Macrobus (packages SSIS), Sql_scripts (scripts de création et d'alimentation), ainsi que le fichier docker-compose.yml et le README.md à la racine.

IV. Infrastructure Docker

4.1 Pourquoi Docker ?

L'utilisation de Docker permet de déployer SQL Server dans un conteneur isolé, garantissant une reproductibilité parfaite de l'environnement sur n'importe quelle machine. Chaque membre de l'équipe dispose ainsi du même serveur SQL Server 2022 avec les mêmes identifiants, évitant les problèmes de version ou de configuration.

4.2 Démarrage du Conteneur

Le lancement de l'infrastructure se fait avec une seule commande depuis le répertoire racine du projet :

```
docker compose up -d
```

```
PS C:\Users\DELL 5420\Documents\Big Data L3\Semestre 1\Data engineering\Projets\Projet_macrobus> docker compose up -d
[+] Running 3/3
 ✓ Network projet_macrobus_default    Created                                    0.1s
 ✓ Volume projet_macrobus_mssql_data  Created                                    0.0s
 ✓ Container moteur_dw_sql            Started                                    0.8s
PS C:\Users\DELL 5420\Documents\Big Data L3\Semestre 1\Data engineering\Projets\Projet_macrobus>
```

Figure 2 — Exécution de docker compose up -d : réseau, volume et conteneur créés

Cette commande crée automatiquement le réseau projet_macrobus_default, le volume de persistance mssql_data, et démarre le conteneur moteur_dw_sql exposé sur le port 1433.

4.3 Vérification dans Docker Desktop

L'interface Docker Desktop permet de confirmer que le conteneur SQL Server est bien actif et consomme les ressources attendues. Le conteneur moteur_dw_sql apparaît avec le statut "en cours d'exécution" et l'image SQL Server est accessible sur le port 1433.

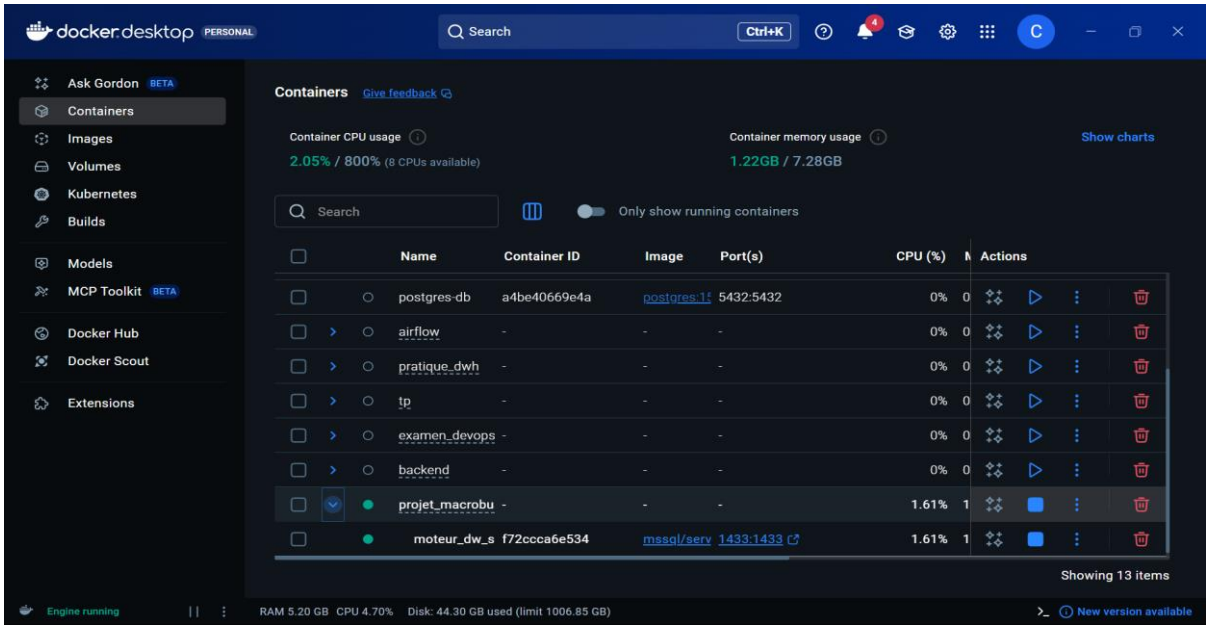

Figure 3 — Docker Desktop : conteneur moteur_dw_sql actif sur le port 1433:1433

4.4 Connexion SSMS au Conteneur

Une fois le conteneur démarré, SSMS se connecte à l'instance SQL Server via localhost,1433 avec l'authentification SQL Server (compte sa). Cette connexion est ensuite réutilisée pour tous les scripts de création et les packages SSIS.

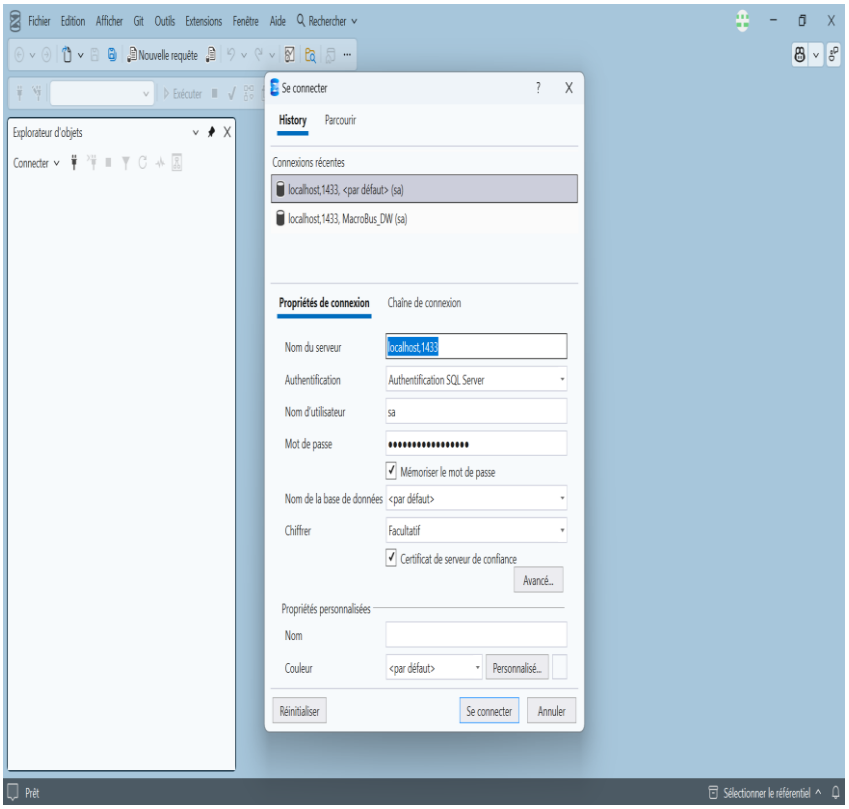

Figure 4 — Fenêtre de connexion SSMS à localhost,1433

V. Base de Données Source (OLTP)

5.1 Rôle du Système Source

La base MacroBus_Source représente le système transactionnel opérationnel (OLTP) de l'entreprise. Elle simule une base de production avec des données brutes issues des opérations commerciales quotidiennes. Quatre tables forment ce modèle relationnel.

5.2 Modèle Relationnel

Table	Clé Primaire	Description
Employes	ID_Employe	Commerciaux avec territoire et filiale
Produits	ID_Produit	Catalogue avec catégorie et prix unitaire
Commandes	ID_Commande	En-têtes de commandes avec date et statut
Details_Commandes	ID_Detail	Lignes de vente : produit, quantité, prix

5.3 Exécution des Scripts SQL

L'ensemble des scripts SQL est exécuté dans SSMS dans un ordre précis. Le script MacroBus_Source.sql crée la structure et insère les premières données de référence. Les scripts d'alimentation génèrent ensuite 20 commerciaux réels, 10 produits et 10 000 commandes couvrant la période 2003–2005.

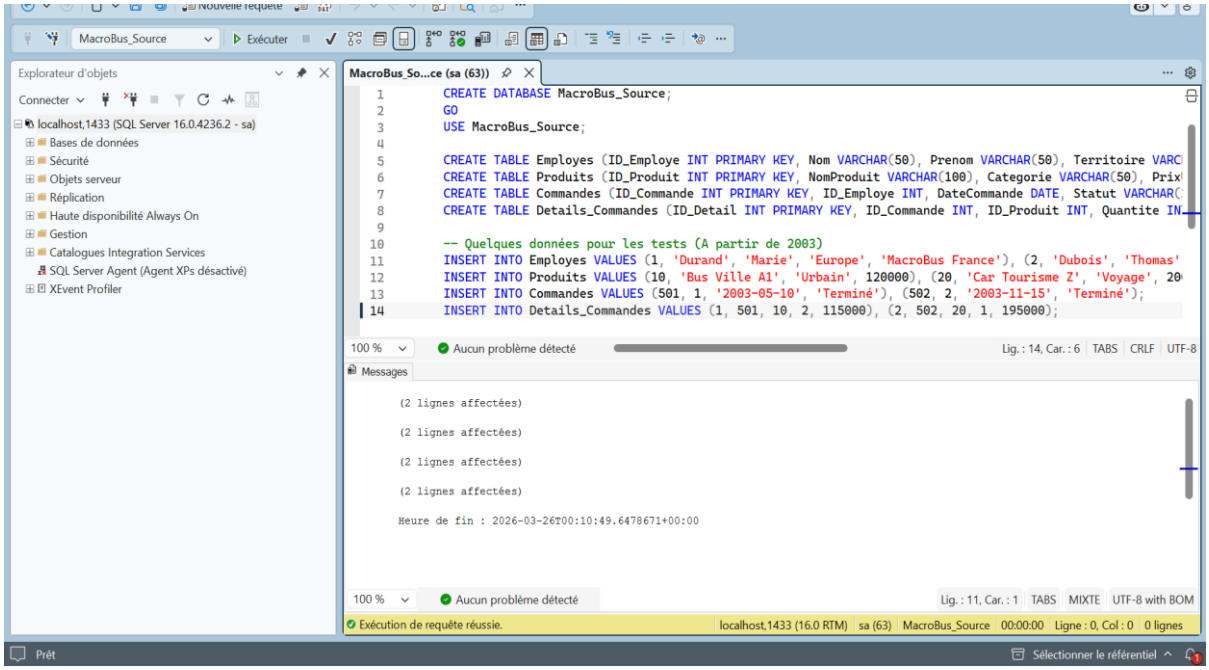

Figure 5 — SSMS : exécution du script MacroBus_Source.sql avec confirmation « Exécution réussie »

5.4 Données Commerciaux (20 profils réels)

Plutôt que d'utiliser des noms génériques, notre équipe a opté pour des profils commerciaux réalistes reflétant la diversité géographique de MacroBus :

```
INSERT INTO Employes VALUES
  (1,  'Ndiaye',   'Abdoulaye',  'Afrique',  'MacroBus Sénégal'),
  (2,  'Leclerc',  'Jean-Pierre','Europe',   'MacroBus France'),
  (3,  'Tremblay', 'Luc',        'Amérique', 'MacroBus Canada'),
  (7,  'Fall',     'Cheikh',     'Afrique',  'MacroBus Sénégal'),
  (8,  'Moreau',   'Lucas',      'Europe',   'MacroBus France'),
  (17, 'Lefebvre', 'Nicolas',    'Europe',   'MacroBus France'),
  -- ... 14 autres commerciaux
```

5.5 Catalogue Produits (10 modèles)

```
INSERT INTO Produits VALUES
  (10,  'CityExpress A1',  'Urbain',             120000.00),
  (20,  'Horizon 3000',   'Voyage',             250000.00),
  (40,  'SchoolMaster XL','Transport Scolaire',  85000.00),
  (50,  'TurboCoach Z',   'Voyage',             320000.00),
  (100, 'InterState GT',  'Voyage',             350000.00),
  -- ... 5 autres produits
```

5.6 Génération des 10 000 Commandes

Un script T-SQL avec une boucle WHILE génère 10 000 commandes réparties aléatoirement entre le 10 janvier 2003 et fin 2005. La première commande est datée précisément du 2003-01-10.

```
DECLARE @i INT = 1, @TotalCommandes INT = 10000;
DECLARE @StartDate DATETIME = '2003-01-10';
WHILE @i <= @TotalCommandes
BEGIN
  SET @CurrentDate =
DATEADD(MINUTE,(@i*120)+ABS(CHECKSUM(NEWID())%60),@StartDate);
  SET @RandomEmp  = (ABS(CHECKSUM(NEWID()) % 20) + 1);
  SET @RandomProd = ((ABS(CHECKSUM(NEWID()) % 10) + 1) * 10);
  INSERT INTO Commandes ...
  INSERT INTO Details_Commandes ...
  SET @i = @i + 1;
END
```

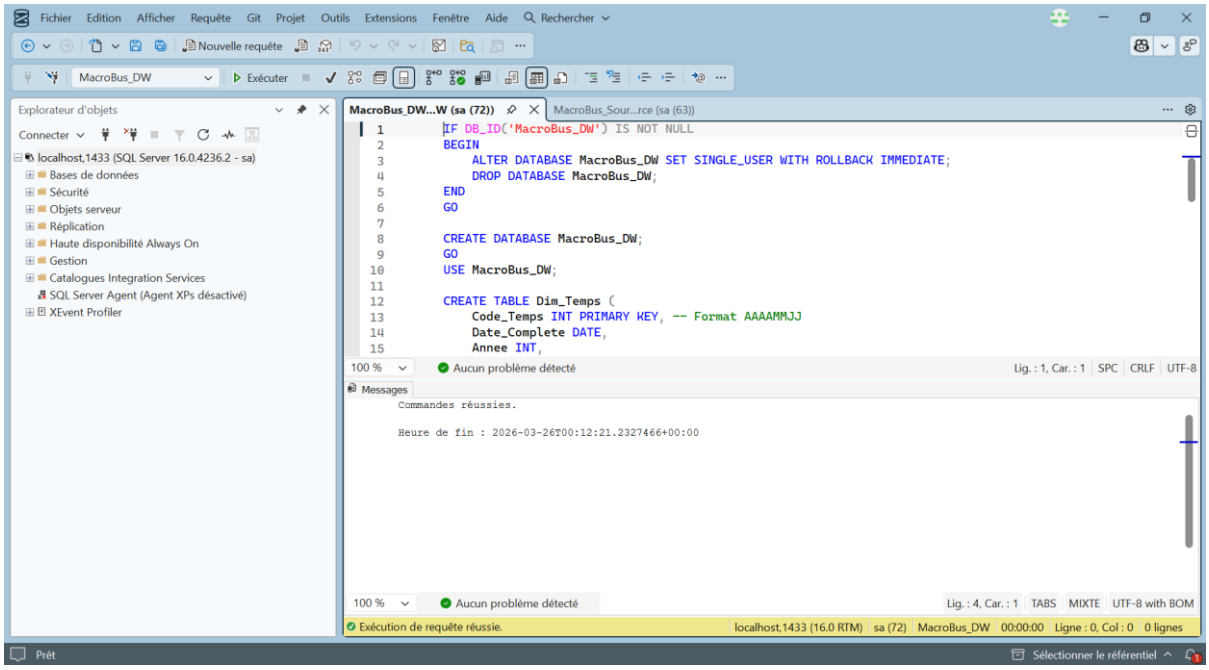

Figure 6 — SSMS : exécution du script MacroBus_DW.sql — « Commandes réussies »

VI. Entrepôt de Données — Schéma en Étoile

6.1 Principe du Schéma en Étoile

L'entrepôt MacroBus_DW est modélisé en schéma en étoile (Star Schema), optimisé pour les requêtes analytiques OLAP. Ce modèle distingue deux catégories de tables :

- Tables de Dimension ("le contexte") : décrivent le Qui, Quoi, Quand — elles sont stables et peu fréquemment modifiées
- Table de Faits ("les chiffres") : centralise les métriques quantifiables — Quantité vendue et Montant total

Fait_Ventes ← Dim_Temps | Dim_Commercial | Dim_Produit

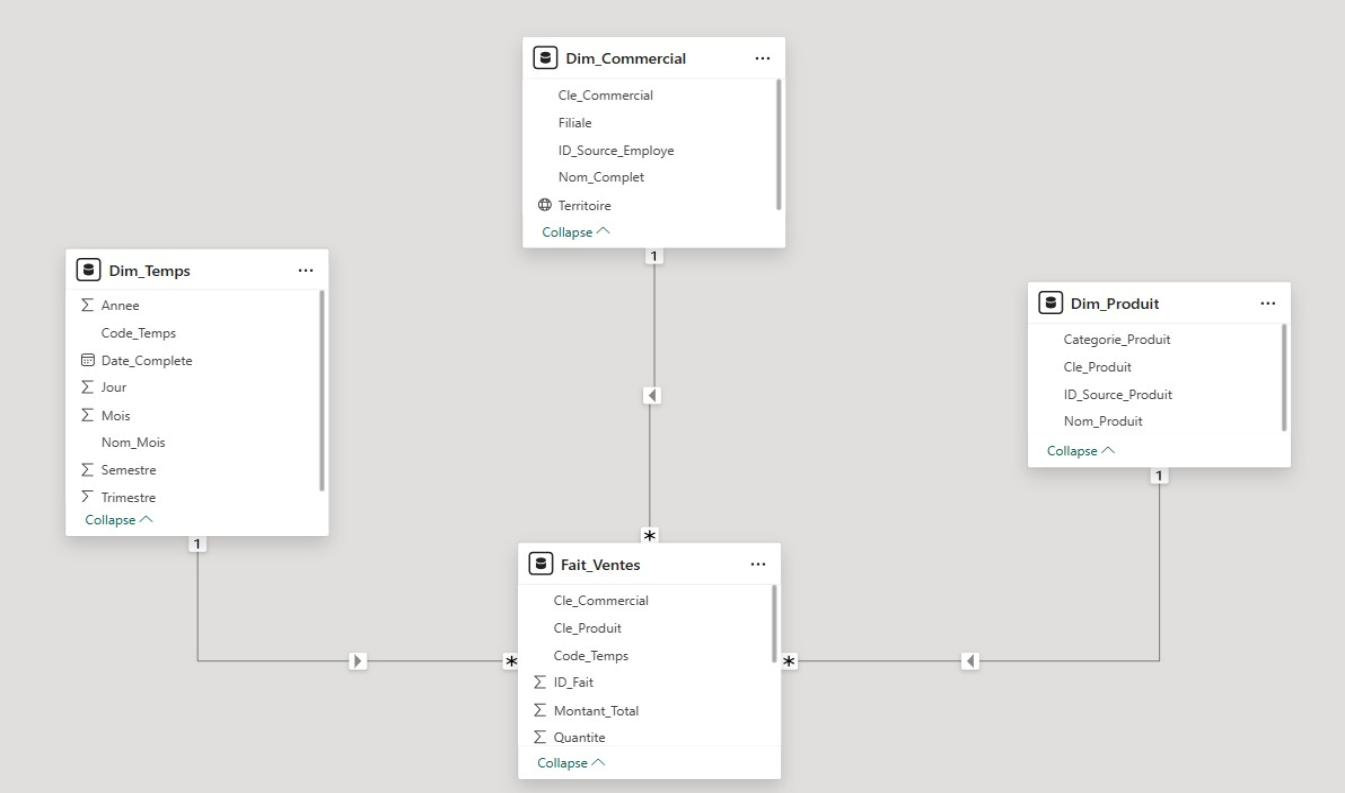

Figure — Modèle en étoile Power BI : relations entre Fait_Ventes et les trois dimensions (Dim_Commercial, Dim_Produit, Dim_Temps)

6.2 Dimension Temporelle — Dim_Temps

La dimension temps est précalculée pour couvrir la période 2003–2028, soit 9 496 jours. Chaque ligne représente un jour avec toutes ses propriétés analytiques (semestre, trimestre, mois, numéro de jour).

```
CREATE TABLE Dim_Temps (
    Code_Temps   INT PRIMARY KEY,  -- Format AAAAMMJJ
    Date_Complete DATE,
    Annee        INT,
    Semestre     INT,
    Trimestre    INT,
```

```
    Mois          INT,
    Nom_Mois      VARCHAR(20),
    Jour          INT
);
```

6.3 Dimension Commerciale — Dim_Commercial

Dimension de type SCD1 pour les commerciaux. La clé technique générée par IDENTITY garantit l'indépendance de l'entrepôt vis-à-vis du système source. On conserve l'ID source (ID_Source_Employe) pour les jointures ETL via les composants Recherche de SSIS.

```
CREATE TABLE Dim_Commercial (
    Cle_Commercial     INT PRIMARY KEY IDENTITY(1,1),
    ID_Source_Employe INT,
    Nom_Complet        NVARCHAR(100),
    Territoire         VARCHAR(50),
    Filiale            VARCHAR(50)
);
```

6.4 Dimension Produit — Dim_Produit

```
CREATE TABLE Dim_Produit (
    Cle_Produit         INT PRIMARY KEY IDENTITY(1,1),
    ID_Source_Produit   INT,
    Nom_Produit         VARCHAR(100),
    Categorie_Produit   VARCHAR(50)
);
```

> ⚠ Le PrixUnitaire n'est pas stocké dans Dim_Produit car le prix varie dans le temps. On stocke le Montant_Total calculé directement dans la table de faits : Quantite × PrixVente.

6.5 Table de Faits — Fait_Ventes

La table de faits est le cœur du schéma en étoile. Elle ne contient que des clés étrangères vers les dimensions et les métriques chiffrées. Les contraintes FOREIGN KEY garantissent l'intégrité référentielle.

```
CREATE TABLE Fait_Ventes (
    ID_Fait         INT PRIMARY KEY IDENTITY(1,1),
    Code_Temps      INT,  -- FK → Dim_Temps
    Cle_Commercial INT,  -- FK → Dim_Commercial
    Cle_Produit     INT,  -- FK → Dim_Produit
    Quantite        INT,
    Montant_Total   DECIMAL(18,2)
);
```

VII. Flux ETL avec SSIS

7.1 Présentation du Pipeline

Le pipeline ETL est le cœur technique du projet. Il assure le transfert, la transformation et le chargement des données depuis MacroBus_Source vers MacroBus_DW. Nous avons utilisé SQL Server Integration Services (SSIS) dans Visual Studio 2022, en structurant le tout en un seul package orchestrant trois tâches séquentielles.

7.2 Control Flow — L'Orchestrateur

Le Control Flow définit l'ordre d'exécution des tâches. L'ordre est imposé par les contraintes de clés étrangères : les dimensions doivent exister avant que la table de faits puisse référencer leurs clés techniques.

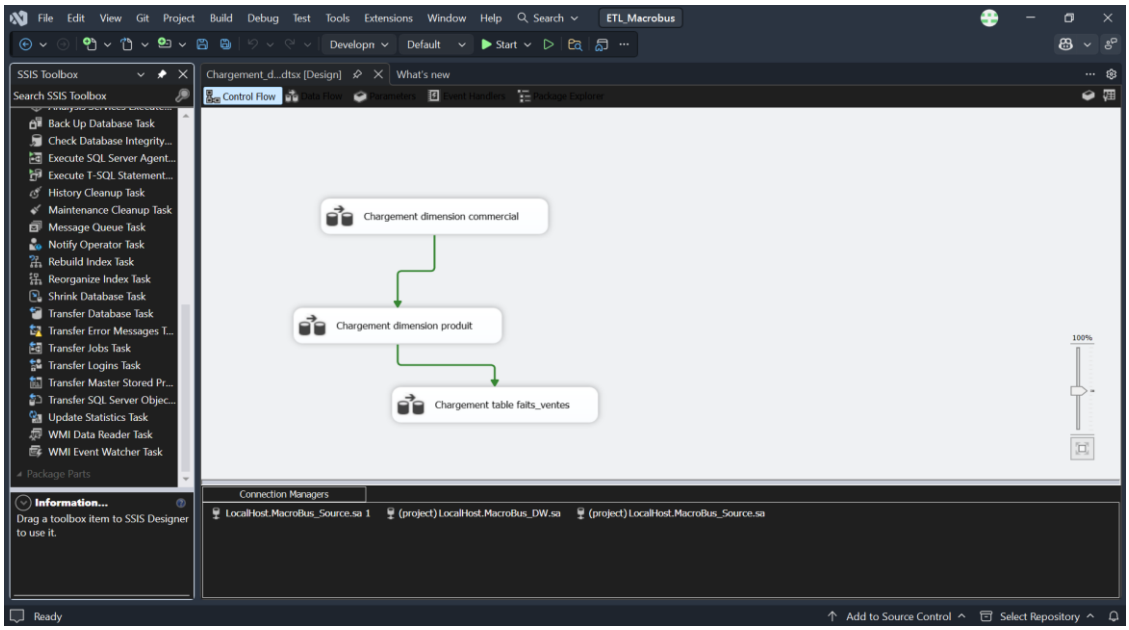

Figure 7 — Control Flow SSIS : ordonnancement des trois tâches de chargement

Les flèches vertes entre les boîtes signifient : "N'exécute la tâche suivante que si la précédente s'est terminée avec succès". Ainsi, la table de fait ne se charge jamais avant que Dim_Commercial et Dim_Produit soient prêtes.

7.3 Gestionnaires de Connexion

Deux connexions OLE DB permettent à SSIS de communiquer avec les deux bases :

Connexion	Cible
LocalHost.MacroBus_Source	Base transactionnelle (source des données)
LocalHost.MacroBus_DW	Entrepôt de données (destination des chargements)

7.4 Chargement des Dimensions

Pour Dim_Commercial, un composant Colonne Dérivée fusionne les colonnes Nom et Prenom de la table source Employes en un champ unique Nom_Complet. Pour Dim_Produit, le mapping est direct entre les colonnes sources et les colonnes de destination.

Source (Employes)	Transformation	Destination (Dim_Commercial)
Nom + Prenom	Colonne dérivée → concat	Nom_Complet
Territoire	Copie directe	Territoire
Filiale	Copie directe	Filiale
ID_Employe	Copie directe	ID_Source_Employe
(auto)	IDENTITY SQL Server	Cle_Commercial

7.5 Data Flow — Chargement de Fait_Ventes

Le chargement de la table de faits est l'étape la plus complexe. Le Data Flow est structuré en chaîne linéaire (la méthode la plus robuste, sans Multicast ni Union All pour éviter les doublons et les valeurs NULL) :

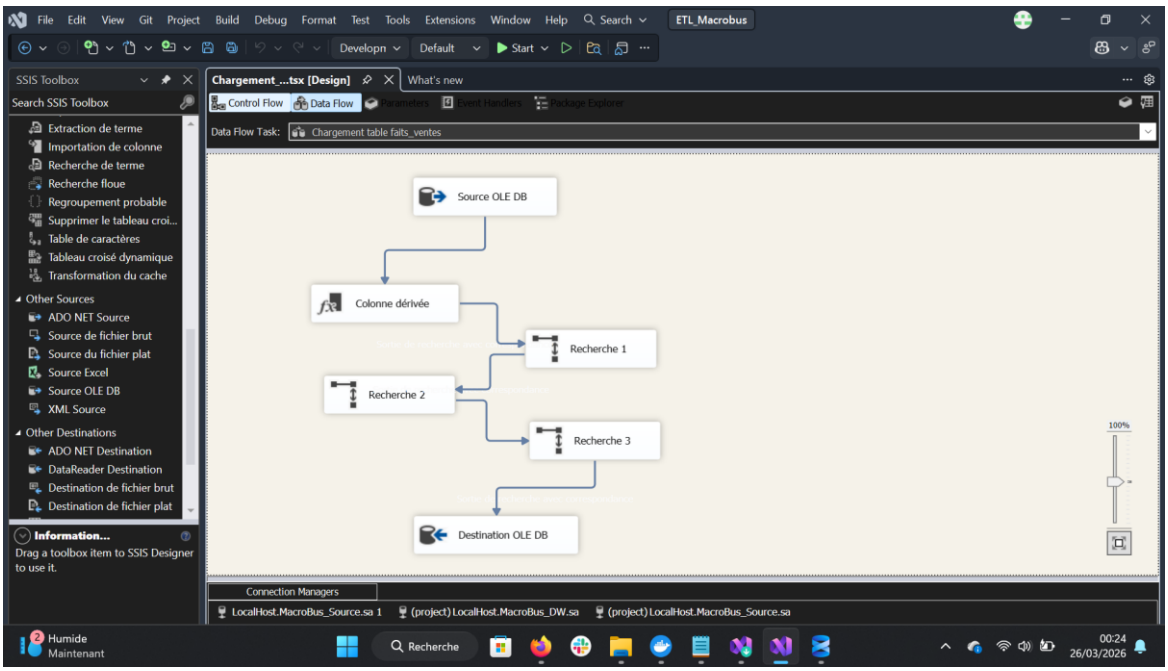

Figure 8 — Data Flow SSIS : pipeline de chargement de Fait_Ventes (Source → Colonne dérivée → Recherche 1/2/3 → Destination)

7.5.1 Source OLE DB

Au lieu de sélectionner une table simple, une requête SQL est utilisée pour joindre Commandes et Details_Commandes et récupérer toutes les informations nécessaires en une seule passe :

```
SELECT C.ID_Employe, C.DateCommande,
```

```
        D.ID_Produit, D.Quantite, D.PrixVente
FROM    Commandes C
JOIN    Details_Commandes D ON C.ID_Commande = D.ID_Commande;
```

7.5.2 Colonne Dérivée — Calcul du Montant

Un composant Colonne Dérivée calcule le Montant_Total en multipliant la Quantite par le PrixVente. Cette opération transforme les données brutes en métriques analytiques directement exploitables.

```
Montant_Total = Quantite * PrixVente
```

7.5.3 Les Composants Recherche (Lookup)

Le composant Recherche est le concept clé du pipeline ETL : il joue le rôle de "traducteur d'IDs". La table de faits ne peut pas stocker les IDs de la source (ex: ID_Employe = 1) — elle doit stocker les clés techniques du DW (ex: Cle_Commercial = 101). Sans ce mécanisme, SQL Server rejette les insertions pour violation de clé étrangère.

Composant	Entrée (Source)	Sortie ajoutée au flux
Recherche 1	ID_Employe → ID_Source_Employe (Dim_Commercial)	Cle_Commercial
Recherche 2	ID_Produit → ID_Source_Produit (Dim_Produit)	Cle_Produit
Recherche 3	DateCommande → Date_Complete (Dim_Temps)	Code_Temps

7.6 Exécution et Résultats

Une fois toutes les connexions configurées sur localhost,1433, l'exécution du package se lance par F5. Les boîtes passent au jaune (en cours d'exécution) puis au vert (succès). Le journal de sortie confirme que 10 000 lignes ont été chargées dans Fait_Ventes.

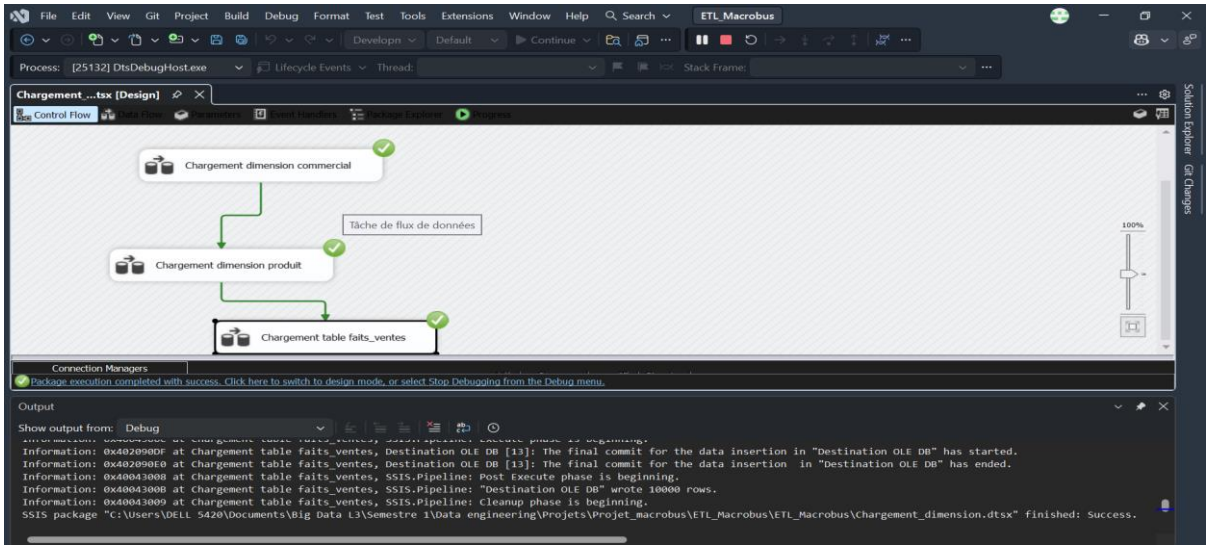

Figure 9 — Exécution ETL réussie : toutes tâches au vert, 10 000 lignes chargées dans Fait_Ventes

> Package execution completed with success. — "Destination OLE DB" wrote 10000 rows.

VIII. Génération et Chargement des Données

8.1 Dimension Temps — Chargement du Calendrier

La table Dim_Temps est la seule dimension qui ne provient pas du système source. Elle est générée par un script T-SQL utilisant une boucle WHILE pour calculer chaque propriété temporelle de 2003 à 2028 (9 496 jours).

Une difficulté rencontrée lors du chargement : la colonne Jour est de type INT, or la fonction DATENAME(WEEKDAY, ...) retourne du texte ("Wednesday"). L'erreur Msg 245 a été résolue en remplaçant DATENAME par DATEPART qui retourne bien un entier.

```
INSERT INTO Dim_Temps VALUES (
    YEAR(@d)*10000 + MONTH(@d)*100 + DAY(@d),  -- Code_Temps
    @d,                                         -- Date_Complete
    YEAR(@d),   CASE WHEN MONTH(@d)<=6 THEN 1 ELSE 2 END,
    DATEPART(QUARTER, @d),  MONTH(@d),
    DATENAME(MONTH, @d),
    DATEPART(WEEKDAY, @d)  -- DATEPART (int) au lieu de DATENAME (varchar)
);
```

⚠ Ordre de nettoyage obligatoire avant rechargement : DELETE FROM Fait_Ventes → DELETE FROM Dim_Temps. L'opération TRUNCATE est bloquée par les contraintes de clé étrangère (Msg 4712).

8.2 Alimentation de la Base Source

Le script final d'alimentation génère 10 000 commandes réparties sur la période 2003–2005, avec des noms de commerciaux réels (Ndiaye Abdoulaye, Moreau Lucas, Lefebvre Nicolas...) et des modèles de bus cohérents avec l'activité MacroBus.

Table	Lignes chargées	Méthode
Employes	20 profils réels	INSERT manuel
Produits	10 modèles	INSERT manuel
Commandes	10 000	Boucle WHILE T-SQL
Details_Commandes	10 000	Boucle WHILE T-SQL
Dim_Temps	9 496 jours	Boucle WHILE T-SQL

IX. Validation Fonctionnelle de l'Entrepôt

Une fois le pipeline ETL exécuté avec succès, la validation de l'entrepôt se fait par des requêtes SQL analytiques directement sur MacroBus_DW. Ces requêtes répondent aux questions métier posées dans le cahier des charges du projet.

9.1 Nombre et Montant Total des Commandes par Territoire

```
SELECT C.Territoire,
       COUNT(F.ID_Fait)        AS Nb_Commandes,
       SUM(F.Montant_Total)    AS Total_Ventes
FROM   Fait_Ventes F
JOIN   Dim_Commercial C ON F.Cle_Commercial = C.Cle_Commercial
GROUP BY C.Territoire;
```

9.2 Top 2 Commerciaux par Nombre de Commandes

```
SELECT TOP 2
       C.Nom_Complet,
       COUNT(F.ID_Fait) AS Nombre_Commandes
FROM   Fait_Ventes F
JOIN   Dim_Commercial C ON F.Cle_Commercial = C.Cle_Commercial
GROUP BY C.Nom_Complet
ORDER BY Nombre_Commandes DESC;
```

9.3 Top 3 Produits au 2ème Trimestre 2003

```
SELECT TOP 3
       P.Nom_Produit,
       SUM(F.Quantite) AS Total_Vendu
FROM   Fait_Ventes F
JOIN   Dim_Produit P  ON F.Cle_Produit  = P.Cle_Produit
JOIN   Dim_Temps   T  ON F.Code_Temps   = T.Code_Temps
WHERE  T.Annee = 2003 AND T.Trimestre = 2
GROUP BY P.Nom_Produit
ORDER BY Total_Vendu DESC;
```

9.4 Montant par Filiale et Catégorie — 2nd Semestre 2003

```
SELECT C.Filiale,
       P.Categorie_Produit,
       SUM(F.Montant_Total) AS Montant_Total
FROM   Fait_Ventes F
JOIN   Dim_Commercial C ON F.Cle_Commercial = C.Cle_Commercial
JOIN   Dim_Produit    P ON F.Cle_Produit    = P.Cle_Produit
JOIN   Dim_Temps      T ON F.Code_Temps     = T.Code_Temps
WHERE  T.Annee = 2003 AND T.Semestre = 2
GROUP BY C.Filiale, P.Categorie_Produit
```

```
ORDER BY C.Filiale, Montant_Total DESC;
```

9.5 Vérification du Comptage Global

```
USE MacroBus_DW;
SELECT 'Dim_Commercial' AS Table_Name, COUNT(*) AS Lignes FROM Dim_Commercial
UNION ALL
SELECT 'Dim_Produit',                  COUNT(*) FROM Dim_Produit
UNION ALL
SELECT 'Dim_Temps',                    COUNT(*) FROM Dim_Temps
UNION ALL
SELECT 'Fait_Ventes',                  COUNT(*) FROM Fait_Ventes;
```

X. Tableau de Bord Power BI

10.1 Connexion et Modélisation

Power BI Desktop se connecte directement à MacroBus_DW (pas à la source). Les quatre tables sont importées et les relations s'appuient sur les clés techniques — jamais sur les IDs sources.

> ⚠ Connecter Power BI uniquement à MacroBus_DW. Utiliser les clés techniques (Cle_Commercial, Cle_Produit, Code_Temps) pour les relations, pas les IDs sources.

10.2 Mesures DAX

Trois mesures de base sont créées pour alimenter les visuels :

```
Chiffre d'Affaires  = SUM(Fait_Ventes[Montant_Total])
Volume Vendu        = SUM(Fait_Ventes[Quantite])
Panier Moyen        = [Chiffre d'Affaires] / [Volume Vendu]
```

10.3 Tableau de Bord — Vue Générale

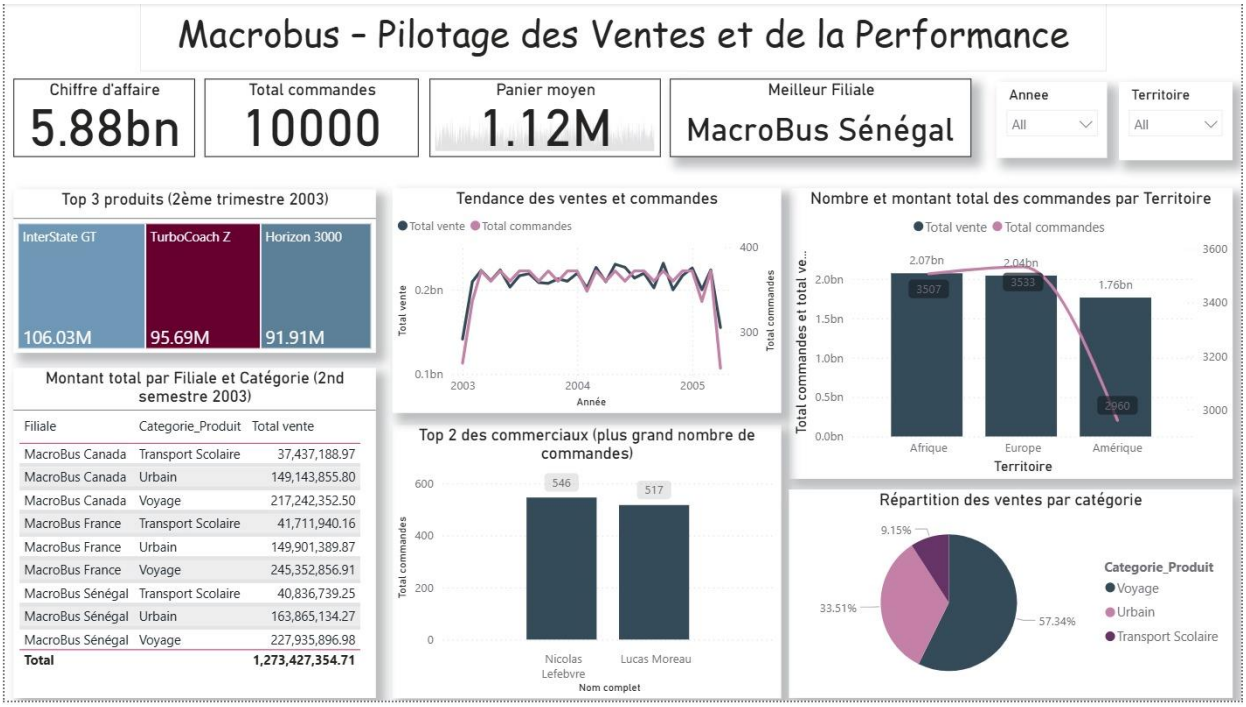

Figure 10 — Tableau de bord Power BI : Macrobus — Pilotage des Ventes et de la Performance

10.4 Indicateurs Clés (KPIs)

KPI	Valeur	Description
Chiffre d'Affaires	5,88 Md FCFA	Somme totale des ventes

Total Commandes	10 000	Nombre de lignes de vente
Panier Moyen	1,12 M FCFA	Montant moyen par commande
Meilleure Filiale	MacroBus Sénégal	Filiale au CA le plus élevé

10.5 Analyses Détaillées

10.5.1 Top 3 Produits — 2ème Trimestre 2003

- InterState GT — 106,03 M FCFA (leader du trimestre)
- TurboCoach Z — 95,69 M FCFA
- Horizon 3000 — 91,91 M FCFA

10.5.2 Ventes par Territoire

- Afrique : 2,07 Md FCFA / 3 507 commandes
- Europe : 2,04 Md FCFA / 3 533 commandes
- Amérique : 1,76 Md FCFA / 2 960 commandes

10.5.3 Répartition par Catégorie

- Voyage : 57,34% — catégorie dominante
- Transport Scolaire : 33,51%
- Urbain : 9,15%

10.5.4 Top 2 Commerciaux

- Nicolas Lefebvre — 546 commandes
- Lucas Moreau — 517 commandes

XI. Conclusion

11.1 Résultats Obtenus

Le projet MacroBus constitue une réalisation complète et fonctionnelle d'une chaîne décisionnelle moderne. L'ensemble des livrables attendus a été produit :

Livrable	Statut	Détail
Base Source OLTP	✓ Réalisé	4 tables, 10 000 commandes 2003–2005
Data Warehouse	✓ Réalisé	Schéma en étoile, 3 dimensions + faits
Pipeline ETL SSIS	✓ Réalisé	1-Click, 0 erreur, 10 000 lignes chargées
Dimension Temps	✓ Réalisé	9 496 jours précalculés (2003–2028)
Dashboard Power BI	✓ Réalisé	7 visuels interactifs + 4 KPIs
Validation SQL	✓ Réalisé	4 requêtes analytiques conformes

11.2 Compétences Acquises

- Modélisation dimensionnelle : distinction OLTP/OLAP, conception en étoile
- T-SQL avancé : procédures, boucles, génération de données volumineuses
- ETL SSIS : Control Flow, Data Flow, Lookup, Colonne dérivée, ordonnancement
- Power BI : import DW, modélisation de relations, mesures DAX
- Infrastructure : conteneurisation Docker, portabilité de l'environnement

11.3 Difficultés Rencontrées

- Erreur Msg 245 : DATENAME retourne du texte — résolu avec DATEPART pour la colonne Jour
- Erreur Msg 4712 : TRUNCATE bloqué par FK — résolu avec DELETE en respectant l'ordre des dépendances
- Doublons dans le cache SSIS : liés au rechargement sans nettoyage préalable des dimensions
- Configuration des connexions SSIS sur de nouveaux postes (localhost,1433 + identifiants Docker)

11.4 Perspectives d'Amélioration

- Implémenter des dimensions SCD Type 2 pour historiser les changements de territoire ou filiale
- Automatiser le rafraîchissement via Power BI Service et une gateway de données
- Ajouter un système de logs SSIS pour le monitoring des chargements en production
- Étendre avec des mesures DAX avancées : YTD (Year-To-Date), croissance MoM

11.5 Organisation de l’équipe et retour d’expérience

La réalisation de ce projet a été le fruit d'un travail véritablement collectif, où chaque membre du groupe a pris en charge une partie bien définie. Les tâches ont été réparties selon les compétences de chacun : la conception de la base de données source (OLTP) et du Data Warehouse a été assurée par certains membres, tandis que d'autres se sont consacrés au développement du pipeline ETL avec SSIS. La création du tableau de bord Power BI a été confiée à une autre partie de l'équipe, et la rédaction du présent rapport a également constitué un livrable à part entière. Chacun a ainsi contribué activement à la réussite du projet, à hauteur de ses responsabilités.

Ce projet nous a également confrontés aux défis concrets du travail en équipe dans un contexte technique distribué. Chaque membre travaillant sur sa propre machine, nous avons régulièrement rencontré des problèmes de compatibilité : un script SQL ou un package SSIS parfaitement fonctionnel sur un poste ne se comportait pas toujours de la même manière sur un autre, en raison de différences de configuration ou d'environnement. Ces situations nous ont obligés à communiquer fréquemment, à documenter nos erreurs et à chercher des solutions ensemble, renforçant ainsi notre cohésion et notre capacité d'adaptation.

Malgré ces obstacles, l'entraide et l'engagement de chacun ont permis d'aboutir à un résultat fonctionnel sur l'ensemble des environnements de travail. Cette expérience nous a rappelé que la réussite d'un projet Big Data ne repose pas uniquement sur les compétences techniques individuelles, mais aussi et surtout sur la qualité de la coordination, la communication et la solidarité au sein du groupe.

> Projet ETL MacroBus réalisé avec succès — L3 Big Data — Dakar Institute of Technology — Mars 2026

Annexes

Annexe A — Scripts SQL

- MacroBus_Source.sql — Création de la base transactionnelle
- MacroBus_DW.sql — Création de l'entrepôt de données
- Alimentation_employéXproduits.sql — Commerciaux et produits de référence
- Alimentation_source_100.sql → Alimentation_source_10000.sql — 10 000 commandes 2003–2005
- chargement_dim_temps_MarcoBus_DW.sql — Calendrier 2003–2028 (version corrigée DATEPART)

Annexe B — Guide de Déploiement Rapide

Pour reproduire le projet depuis zéro sur un nouveau poste :

- 1. Installer : Docker Desktop, SSMS, Visual Studio 2022(2026 actus) + SSIS, Power BI Desktop
- 2. Lancer : docker compose up -d dans le répertoire du projet
- 3. SSMS : exécuter les 5 scripts SQL dans l'ordre indiqué
- 4. Visual Studio : ouvrir ETL_Macrobus.slnx, configurer les connexions sur localhost,1433
- 5. F5 : exécuter le flux ETL — attendre que toutes les tâches passent au vert
- 6. Power BI : ouvrir le .pbix et actualiser la connexion sur MacroBus_DW

Annexe C — Résolution des Erreurs Fréquentes

Erreur	Cause	Solution
Row yielded no match (Recherche 3)	Dim_Temps vide ou dates absentes	Relancer chargement_dim_temps.sql
Msg 245 : conversion nvarchar → int	DATENAME retourne du texte	Remplacer par DATEPART(WEEKDAY,...)
Msg 4712 : TRUNCATE bloqué	Contrainte FK active	DELETE FROM Fait_Ventes d'abord
Doublons dans le cache SSIS	Dimensions rechargées sans nettoyage	DELETE Dim avant relance SSIS
Croix rouges Visual Studio	Connexions pointent ancien serveur	Reconfigurer localhost,1433 dans Connection Managers